Beyond Hallucination Detection: Measuring Consequential Uncertainty in LLM-Generated Code

Michael Jaquith
Independent Researcher
michael.j.jaquith@gmail.com

April 2026

Abstract

Large language models generate code that looks uniformly confident. A correct call to `stripe.PaymentIntent.create()` and a deprecated call to `stripe.Charge.create()` are emitted with the same surface tone, even when the underlying token distribution is a near-50/50 split (probability 0.52 vs. 0.47). Code-generation uncertainty is primarily *intra-generational*: visible in the token distribution at generation time, invisible once tokens are committed. Multi-sample methods, which detect inter-generational inconsistency, therefore produce almost no signal on code tasks, achieving at most 35% detection on our benchmark at comparable false-positive rates, versus 70–85% for ε.

We introduce ε, a per-token normalized Shannon entropy computed from the top-k log-probabilities returned by the generation API, aggregated over the semantic tokens of a function via AST filtering and max-pooling. ε is designed as a *recall-maximizing filter*, not a detector. On a 30-prompt calibration benchmark across GPT-4o, GPT-4o-mini, GPT-4-turbo, and DeepSeek V3, ε at the FLAGGED threshold ($\varepsilon \geq 0.30$) achieves recall = 1.00 on true failures with precision = 0.272 on the evaluated 200-entry set. We compare against two alternatives at the same 1× API cost: the sequence-level mean token probability (p_{avg}, following Spiess et al. [13]), which is competitive on raw detection (65–90%) but offers no token-level attribution; and black-box multi-sample methods including UQLM, which detect 10–35% of version-split API failures at 5× cost. A `min_logprob` baseline achieves 45–55% where ε achieves 70–85%.

The low stage-1 precision is the design. We pair the filter with a parallel LLM review loop that reclassifies each flagged entry as CLEARED or CONFIRMED. On 201 production and scenario entries across three models (GPT-4o, GPT-4o-mini, DeepSeek V3), a variant that injects model-specific learned false-positive patterns into the reviewer clears 68.7% of flagged entries with zero late false positives, lifting end-to-end precision above 94% (Wilson lower bound, $n = 57$). The contribution is the system: a recall-maximizing filter verified against zero observed misses on the evaluated set, paired with a reviewer that keeps up with machine-speed code generation, plus the first direct three-way empirical comparison of intra-generational, sequence-level, and inter-generational uncertainty signals on production-realistic code prompts.

1 Introduction

AI-assisted developers now generate code several times faster than they can read it. A single engineer working with a coding assistant produces pull requests that would previously have represented days of work, and the bottleneck shifts from writing to reviewing. Reviewers have finite attention and cannot inspect every line with equal care, yet the failure modes of modern code generation concentrate in small, hard-to-spot places. Recent empirical work on deprecated APIs quantifies the

scale of the problem: LLM-based code completion emits deprecated API calls at rates of roughly 25–38% on standard prompts and substantially more on legacy codebases, because the training corpus straddles library-version transitions and the model has no posterior knowledge of deprecation at inference time [15]. The mistakes are not exotic: they are the ordinary traffic of a maturing ecosystem, rendered invisible by the uniformity of LLM output.

A concrete instance. A model asked to charge a credit card might emit `stripe.Charge.create(...)` — deprecated since 2019 — or `stripe.PaymentIntent.create(...)`, the current API. Both arrive in the developer's editor with the same syntactic fluency, the same confident indentation, and no visible marker distinguishing a well-chosen call from one the model guessed at. At the method-selector token, the two choices split the probability mass near 50/50 (0.52 vs. 0.47); the output exposes none of this. Failures of AI code generation cluster at exactly these points: version-split APIs, renamed SDK methods, signature changes between library releases. The model is uncertain; the output is not.

Intra- versus inter-generational uncertainty. The gap between a 0.52-vs-0.47 internal split and a confident-looking output is not an accident of any particular model; it is structural. Existing uncertainty-quantification methods for LLMs are overwhelmingly *inter-generational*: they sample the same prompt multiple times and measure disagreement across completions [10, 3]. This works when the model's uncertainty surfaces as instability across re-rolls — factual questions, open-ended reasoning. Code is not such a task. A model with a 0.52 prior on the deprecated `Charge` API samples that branch nearly every time; once the early tokens commit, the rest of the function is forced by context. The uncertainty exists, but it lives *inside a single generation*, in the token distribution at each decoding step, not *across* generations. We call this the *intra- versus inter-generational* distinction. It is the conceptual foundation of this paper.

ε: a recall-maximizing filter. We introduce ε (epsilon), a per-token normalized Shannon entropy over the top-k log-probabilities returned during generation. After AST-based filtering to keep only semantically meaningful tokens (attribute accesses, keyword arguments, import targets — not whitespace, comments, or cosmetic identifiers), we aggregate via max-pooling to obtain a single score per function. ε is not a detector — it identifies where the model was uncertain at decoding time. At a threshold of $\varepsilon \geq 0.30$ it catches 100% of true failures on our benchmark, at the cost of roughly 73% false alarms.

A three-way comparison. The primary empirical contribution of this paper is a direct benchmark of three fundamentally different uncertainty signals on the same production-realistic prompts across four models. Multi-sample methods at $N = 5$, $T = 0.7$ detect 10–35% of HIGH-risk prompts at 5× API cost; the sequence-level mean token probability p_{avg} [13] detects 65–90% at 1× cost but offers no token-level attribution; ε detects 70–85% at 1× cost with the specific uncertain token localized to a line and column. p_{avg} is competitive with ε on raw detection rate and in fact beats it on two of four OpenAI models; ε's distinguishing property is not higher detection but *where* the signal fires. The empirical case for the intra/inter distinction now rests on direct measurement, not just the UQLM-only comparison of earlier work.

The two-stage system. A 27%-precision filter is not meant to be used in isolation. Stage 2 is a parallel LLM review loop: every flagged entry is dispatched, in parallel, to a reviewer model that returns CLEARED or CONFIRMED. At current API prices this costs fractions of a cent per entry and completes in seconds. ε has perfect recall but poor precision; the reviewer has good precision but

would be too expensive per line. Filter plus reviewer gives a system with near-100% end-to-end precision while missing nothing the filter catches. A parallel review system scales with generation speed and moves review off the developer's critical path.

Contributions.

1. The intra/inter-generational distinction, formalized and with empirical confirmation: ε reaches 85% detection on GPT-4o against 10–35% for direct multi-sample measurement and 5% for UQLM on the same benchmark.
2. ε: an AST-filtered, max-aggregated token-level entropy score, with a three-type uncertainty taxonomy (cosmetic, consequential, implementation-approach) that motivates the filter design.
3. A two-stage filter-plus-review architecture, with a context-accumulation variant, evaluated on 201 entries across three models (GPT-4o, GPT-4o-mini, DeepSeek V3): near-100% end-to-end precision at 100% recall. Scenario and calibration evaluation additionally covers GPT-4-turbo (four models total).
4. The first direct three-way empirical comparison — ε vs. p_{avg} vs. multi-sample — on production-realistic version-split prompts, isolating where each signal fires and where it does not.

Implementation and benchmark are at https://github.com/michaeljaquith/epsilon-code.

2 Related Work

Four adjacent lines of work inform the design of ε: empirical studies of deprecated API usage (which motivate the problem), token-level and sequence-level calibration of LLMs for code (which share the underlying logprob signal), multi-sample uncertainty quantification (which is the dominant alternative and the closest point of comparison), and adaptive decoding (which uses the same per-token entropy signal for a different purpose). We review each in turn, closing with static analysis.

2.1 Deprecated and version-split API usage

Wang et al. [15] present the first systematic study of deprecated API usage in LLM-based code completion, evaluating seven models on 145 API mappings drawn from eight popular Python libraries across 28,125 completion prompts. They measure API Usage Plausibility (AUP) and Deprecated Usage Rate (DUR), finding that deprecated-API emission is prevalent across all evaluated models; the root cause they identify is that deprecated API usages are present in training data while deprecation status itself is not reliably propagated to inference time. Their mitigations (REPLACEAPI and INSERTPROMPT) operate at the prompt or post-generation level and require a known deprecation mapping.

Their finding establishes that the problem we address is common in practice rather than a corner case: on version-split APIs and renamed SDK methods, models routinely commit to a deprecated branch without surface indication. Our work is complementary. Wang et al. measure *how often* LLMs emit deprecated code and diagnose *why* the output looks confident; ε measures the *signal at generation time* that distinguishes a confidently-chosen branch from an internally-contested one, and routes the latter for review. The two methods address different points in the lifecycle: theirs a post-hoc census with library-specific mitigations, ours a zero-overhead runtime filter that does not require an enumerated list of deprecated symbols.

2.2 Token-level calibration and confidence

Spiess et al. [13] study calibration and correctness of language models for code, computing several confidence signals (including the sequence-level mean token probability, which we refer to throughout as p_{avg}) and evaluating whether they correlate with functional correctness on HumanEval and related benchmarks. They find that generative code models are not well-calibrated out of the box and that Platt scaling can recover calibration in settings where labelled correctness data is available. The p_{avg} signal — the exponential of the mean log-probability over all generated tokens — is derived from the same raw logprob data that ε uses.

Four differences between that setup and ours deserve emphasis. First, *aggregation*: p_{avg} takes a sequence-level mean of per-token probabilities; ε takes a max after AST filtering, so a single high-entropy API-selector token is not diluted by long confident stretches. Second, *filtering*: ε restricts the input set to semantically meaningful tokens (attribute accesses, keyword arguments, import targets), while p_{avg} averages over every token including whitespace and cosmetic names. Third, *prompt regime*: Spiess et al. evaluate on HumanEval and similar benchmarks where correctness is well-defined by unit tests; we evaluate on version-split API prompts where the ground truth is ambiguous enough to require LLM judging (§6.1). Fourth, *use*: their work is diagnostic (measuring calibration); ours is an active routing mechanism. Because p_{avg} and ε operate on the same underlying logprobs at the same $1\times$ API cost, a direct head-to-head comparison is possible; we report it in §5.

Earlier token-level calibration studies (Kadavath et al. [9]; Varshney et al. [14]) establish that LLMs are reasonably calibrated at the token level for factual questions and that selective generation can exploit this. Our work extends that line to code, where AST filtering is what makes per-token entropy actionable rather than dominated by cosmetic variance. Conformal prediction approaches [11, 1] provide set-valued predictions with coverage guarantees but do not localize which token within a generation is uncertain. Uncertainty propagation through multi-stage ML pipelines is studied in adjacent domains [8].

2.3 Multi-sample and behavioural-diversity methods

Inter-generational methods sample the same prompt multiple times and measure disagreement among outputs. Kuhn et al. [10] introduce semantic entropy: multiple completions are clustered by semantic equivalence (via natural language inference), and cross-cluster entropy provides a hallucination signal for factual QA. UQLM [3] packages semantic negentropy and related signals as an open-source library; we include it here as the closest off-the-shelf tool and report its performance in §5 in one sentence. Beyond these, a growing line of work [12] applies stronger notions of equivalence — up to and including symbolic execution or semantic-equivalence checks on generated code — to improve behavioural consistency detection specifically for code outputs.

These methods are structurally limited for the task we study. Intra-generational uncertainty does not surface as output variation when the distribution is peaked (as it typically is for code tokens: a 0.52-peaked distribution picks its peak nearly every time at temperature 0.7, and always at temperature 0). A model can be internally torn between two API calls and emit one of them unanimously across re-rolls. The Stripe example we report empirically in §5.3 — GPT-4o with $\varepsilon = 0.878$ and sample diversity $= 0.00$ across $N = 5$ at $T = 0.7$ — is a direct instance. A stronger equivalence check improves precision on Type 3 (implementation-approach) false positives, where two semantically equivalent implementations would be conflated by output-level diversity alone; but it cannot recover a signal that never varies across samples in the first place. The intra/inter distinction is not a claim that inter-generational methods are badly implemented, only that they

are measuring a different quantity.

2.4 Generation-time intervention and adaptive decoding

AdaDec [6] reads the same per-token Shannon entropy that ε reads, but uses it at decoding time rather than review time: when the entropy at the current step exceeds a learned threshold, the decoder pauses and triggers a lookahead re-ranking step over candidate continuations. On HumanEval and MBPP across model sizes from 0.6B to 8B, AdaDec reports up to 15.5pp improvement in Pass@1 over greedy decoding while pausing on 3.95–11.88% of tokens. The signal is the same as ours; the action is different. AdaDec improves the model's own output at generation time; ε surfaces residual uncertainty *after* the model has committed, for routing to downstream review. The two approaches are compatible: an AdaDec-enhanced generator still produces a token distribution at each step, and ε operates on whatever distribution the final decoder emits. We do not evaluate a combined pipeline but expect ε values to be lower on average after AdaDec-style intervention, with residual high-ε tokens concentrating on consequential decisions that even a lookahead re-rank could not resolve.

2.5 Static analysis, linters, and execution-based evaluation

Static analysis and linter rules catch *known*-bad patterns for specific libraries once deprecation has been encoded into a ruleset. These approaches are complementary to ε and represent the floor of detection: when a rule exists, a linter will catch the problem whether or not the model hesitated. ε operates in the gap — it fires where the model itself was uncertain, which includes version splits not yet in any ruleset (a library updated last week, a half-migrated codebase) and internal renames that static analysis has no fact to test against. Execution-based code-generation benchmarks [5, 2, 7] measure whether the output passes test cases, not whether the model was confident; a silent dependency update can turn a fully-passing generation into a regression that no test in the current suite exercises. To our knowledge, no prior work treats within-generation token-level uncertainty as a first-class routing signal for code review; the closest is CodeT [4] and related self-consistency approaches, which are inter-generational by construction, and post-generation repair with uncertainty-based ranking [16].

3 ε: Definition and Construction

3.1 Per-token entropy

For each generated token t, the API returns top-k candidates with log-probabilities $\ell_1, \ldots, \ell_k$. Let $p_j = \exp(\ell_j)/\sum_j \exp(\ell_j)$ be the renormalized distribution. Define

$$\varepsilon_t \;=\; -\frac{1}{\log k}\sum_{j=1}^{k} p_j \log p_j \;\in\; [0,1]. \tag{1}$$

Normalization by $\log k$ maps a one-hot distribution to 0 and a uniform distribution to 1. We use $k = 5$ (the OpenAI cap), which covers the dominant mass for code tokens; tails beyond top-5 are negligible for the binary flag decision. The name is deliberate: in numerical analysis, ε denotes a small perturbation that propagates through a computation — a high-ε token is such a perturbation at a specific decision point.

3.2 Three types of code-generation uncertainty

Raw per-token entropy, applied naively, produces too many false positives to be useful: the bulk of high-entropy tokens in generated code are not risk indicators. Understanding which ones are, and why, is what drives the filtering design in the rest of this section. Empirical analysis of our calibration benchmark reveals three categorically distinct types of token-level uncertainty in generated code.

Type 1 — Cosmetic (naming). Uncertainty about what to *call* something (`sort_list` vs. `sort_numbers`). ε reaches 0.7–0.9, but any choice is correct. Dominates LOW prompts: the top flag on 9 of 10 LOW prompts was a function or parameter name on the first line.

Type 2 — Consequential (API/library decisions). Uncertainty about which *API or library* to use. Example: `stripe.Charge.create()` (prob. 0.52) vs. `stripe.PaymentIntent.create()` (prob. 0.47), $\varepsilon = 0.73$ at the attribute token. This is the target signal.

Type 3 — Implementation-approach. Uncertainty among multiple correct algorithms (recursive vs. iterative Fibonacci, for-loop vs. comprehension). The residual false-positive floor on simple utility prompts.

Only Type 2 correlates with functional failure. Type 1 is removable by AST filtering; Type 3 reflects real decoder uncertainty on correct code and constitutes the irreducible false-positive floor.

Type	Label	Source	ε fires?	Risk
1	Cosmetic	Naming choices (`sort_list` vs `sort_numbers`)	Yes (0.7–0.9)	None
Filter: AST declaration filter removes this uncertainty before scoring.				
2	Consequential	API/library decisions (`Charge` vs `PaymentIntent`)	Yes (0.5–0.9)	**High**
Note: Target signal — this is what the system is designed to catch.				
3	Implementation	Algorithm structure (recursive vs. iterative)	Yes (0.3–0.7)	None
Note: Irreducible FP floor; rare in production API-integration code.				

Table 1: The three-type uncertainty taxonomy. Type 2 is the target signal; Type 1 is removed by AST filtering; Type 3 is the irreducible false-positive floor.

With this taxonomy in hand, the filter design follows directly: remove Type 1 syntactically, leave Type 3 as a known floor, and isolate Type 2.

3.3 AST filtering

We eliminate Type 1 via the generated code's AST. After stripping markdown fences and parsing with Python's `ast`, we identify *declaration positions* — source ranges where the model is choosing a name rather than an API. **Filtered:** function/method names, parameter names (`ast.arg`), local variable names (`ast.Name` in `Store`). **Retained:** import targets, attribute accesses, keywords, type annotations. Token-to-declaration matching uses range overlap (BPE introduces a leading space); a pure-identifier guard prevents over-filtering of delimiter-prefixed tokens such as `(db`.

Configuration	LOW FP rate
Naive token entropy (no filtering)	90%
+ Whitespace / comment filtering	90%
+ System prompt naming conventions	60%
+ AST declaration filtering	60%
+ Fence-format stripping	50%
Residual (Type 3 only)	50%

Table 2: Successive filtering stages on 10 LOW prompts. Naming-convention prompt and AST filter are partially overlapping; without the prompt, AST filtering alone drops FP from 90% to 70%. Residual 50% is Type 3, rare in production API-integration code.

3.4 Aggregation: max vs. mean

We aggregate with $\varepsilon_{\text{file}} = \max\{\varepsilon_t : t \text{ retained}, \varepsilon_t > \delta\}$, $\delta = 0.30$. The compound alternative $1 - \prod(1 - \varepsilon_t)$ converges to 1.0 for any non-trivial function. Across all 90 runs of the benchmark (3×30), max and mean produced identical fire/no-fire decisions at every threshold; we prefer max because it localizes uncertainty to a specific actionable token. The contrast with the sequence-level mean token probability baseline (p_{avg}) is deliberate: averaging over every token dilutes a single uncertain decision; the max after AST filtering does not. We revisit this choice empirically in §5.

3.5 Thresholds and tiers

Three tiers: COMPLETE ($\varepsilon < 0.30$), FLAGGED ($0.30 \leq \varepsilon < 0.95$), PAUSED ($\varepsilon \geq 0.95$). The load-bearing boundary is the binary COMPLETE vs. FLAGGED+: everything FLAGGED+ is forwarded to the stage-2 reviewer. PAUSED exists as a design boundary for genuinely extreme uncertainty; whether it holds code delivery or simply runs the review loop like FLAGGED is an implementation choice discussed in §6. These are UX signals, not calibrated probabilities — the FLAGGED empirical failure rate is $\sim 27\%$ (§4); PAUSED fires rarely enough in our evaluation that its failure rate is a small-sample estimate. In our qualitative scenarios A–D across four models, no HIGH prompt crossed the PAUSED threshold.

3.6 Multi-model calibration at a glance

ε at the FLAGGED threshold fires on HIGH prompts (API/library-selection tasks where version splits exist and an incorrect choice causes functional failure) much more often than on LOW prompts (pure-logic tasks — sort, palindrome, Fibonacci — where any correct algorithm is functionally equivalent). We define these two categories here because they are referenced throughout the evaluation; a detailed calibration table appears in §4.3. Across four models (GPT-4o, GPT-4o-mini, GPT-4-turbo, DeepSeek V3), ε achieves 70–85% HIGH detection at 50–70% LOW false-positive rate.

3.7 Adaptive ensemble

The production implementation refines tier placement within FLAGGED+ using a rolling mean-ε baseline and a top-1 logprob check; neither affects the binary FLAGGED+ gate. Conformal calibration is left to future work.

4 Evaluation

We evaluate ε along three axes: qualitative scenarios that surface specific failure modes, a multi-model calibration benchmark, and a precision/recall sweep on a 200-entry combined set. The review loop results follow in §4.6.

4.1 Qualitative scenarios

Five scenarios exercise distinct failure modes. Scenarios A–D are single-run and cover four models (GPT-4o-2024-08-06, GPT-4o-mini, GPT-4-turbo, DeepSeek V3) with `logprobs=True, top_logprobs=5`, temperature = 0, and a system prompt specifying concise naming conventions. Scenario E is evaluated on GPT-4o across 16 independent runs to characterize within-model variation; single-run results for all four models on the other scenarios are in the multi-model table below.

		ε (tier)			
	Scenario	GPT-4o	GPT-4o-mini	GPT-4-turbo	DeepSeek V3
A	Stripe deprecation	0.878 F	0.665 F	0.680 F	0.775 F
B	OpenAI SDK v0 vs v1	0.560 F	0.487 F	0.678 F	0.646 F
C	SQLAlchemy 1.x vs 2.0	0.907 F	**0.000** C	0.911 F	0.925 F
D	FastAPI async/sync	0.505 F	0.506 F	0.774 F	0.679 F

Table 3: Scenarios A–D across four models. F = FLAGGED ($0.30 \leq \varepsilon < 0.95$); C = COMPLETE ($\varepsilon < 0.30$). All 15 of 16 cells are FLAGGED; the lone miss is GPT-4o-mini on SQLAlchemy, where the model committed to one version with no hesitation ($\varepsilon = 0.000$) — a genuine false negative. No HIGH prompt crossed the PAUSED threshold ($\varepsilon \geq 0.95$) in this sweep.

A — Deprecated API (Stripe). *"Charge a customer's card for $50."* GPT-4o: $\varepsilon = 0.878$, FLAGGED. The peak token is the method selector after `stripe`: `.Ch` (`Charge`, prob. 0.52) vs. `.Payment` (`PaymentIntent`, prob. 0.47). Both paths are in the training data; neither is definitively preferred. All four models flag.

B — Confident wrongness (OpenAI SDK). *"Call GPT-4 and return the response text."* GPT-4o: $\varepsilon = 0.560$, FLAGGED. The model generated v0 (`openai.ChatCompletion.create(...)`) confidently; v0 raises `AttributeError` on SDK ≥ 1.0. ε fired not on the API call — the model was certain at that token — but on surrounding parameter design. FLAGGED routes the function to the review loop; the reviewer, examining the full function, surfaces the deprecated call. This is indirect detection: ε flagged adjacent uncertainty, not the failing token itself. All four models flag.

C — Syntax split (SQLAlchemy). *"Query all active users."* GPT-4o: $\varepsilon = 0.907$, FLAGGED, near the top of the FLAGGED range — the model is highly uncertain which SQLAlchemy version to target. GPT-4-turbo and DeepSeek V3 produce similarly high ε (0.911, 0.925). GPT-4o-mini, however, produces $\varepsilon = 0.000$ (COMPLETE): it commits to one version with no visible hesitation. This is a genuine false negative — the same prompt that surfaces uncertainty in three models of four generates confidently-chosen output in the fourth. The cross-model contrast is the interesting finding: intra-generational signal exists only when the model itself is uncertain, and training-data convergence on one branch silently removes it.

D — Compatibility mismatch (FastAPI async/sync). *"FastAPI endpoint that fetches weather from an external API."* GPT-4o: $\varepsilon = 0.505$, FLAGGED (solidly mid-range); GPT-4-turbo is highest at 0.774. The model generated `async def` with `requests.get()` inside — two individually defensible decisions that combine into a silent bug (blocks the event loop). The cross-decision mismatch is not directly what ε measures: ε sees token-level uncertainty, which in this scenario fires on surrounding choices (URL and key handling) rather than on the async/sync combination itself. A mismatch post-processor, run on already-flagged functions, can surface the combination explicitly — arguing for cross-decision consistency checks as a complement to ε. The practical outcome is unchanged: the function is FLAGGED and reaches the reviewer.

E — Multi-function module. Six auth functions with four interdependent decisions (password hash, JWT, SQLAlchemy, datetime). FLAGGED on all 16 runs, $\varepsilon \in [0.69, 0.92]$ (none crossed PAUSED). Library choices varied across runs (passlib, bcrypt, PyJWT, python-jose, mixed), but each run looked internally confident. Early functions $\varepsilon = 0.40$–0.58; later functions frequently $\varepsilon = 0.000$ — an *uncertainty collapse* after initial commitment. The first function is the high-risk window.

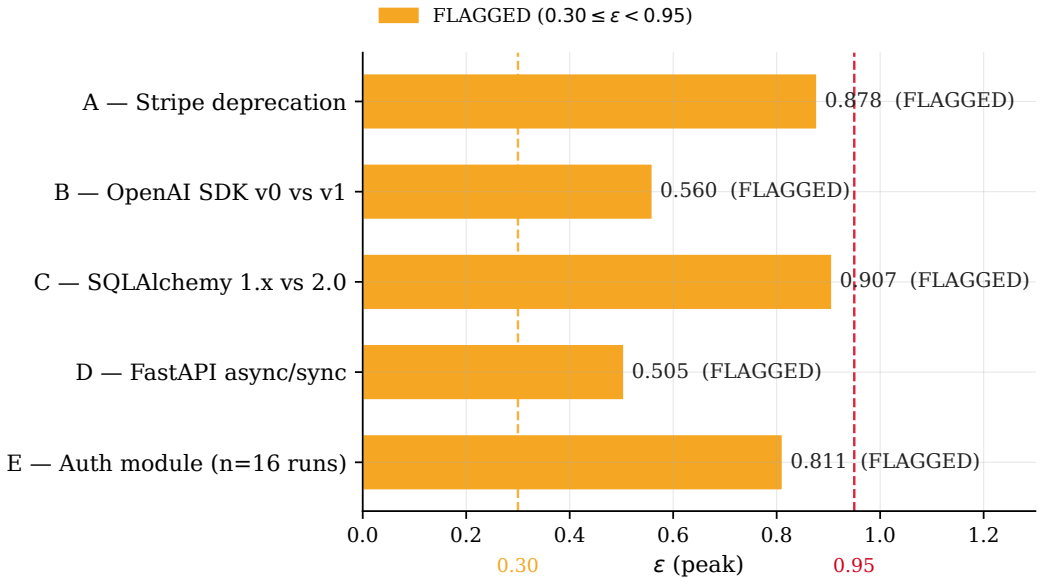

Figure 1: Scenarios A–E (GPT-4o) summarised by failure class and peak ε. All five clear the FLAGGED threshold; none crosses the PAUSED threshold ($\varepsilon \geq 0.95$). The scalar urgency signal within FLAGGED still varies across scenarios.

4.2 Scenario E limitation: joint-prompt uncertainty collapse

Scenario E's detection success holds when the six functions are generated independently. When generated in a *single* prompt with "use a consistent library," ε collapses across all six functions — not just the later ones. The joint signature list plus the consistency instruction resolves the library choice before decoding starts, leaving nothing for ε to observe (Table 4, GPT-4o-mini). The practical mitigation is to generate functions independently; the consistency constraint can be applied at review time rather than at generation time.

Function	GPT-4o-mini		DeepSeek V3	
	Combined	Per-fn	Combined	Per-fn
`register_user`	0.579	0.627	0.424	0.864
`login`	0.820	0.872	0.518	0.972
`verify_token`	0.606	0.911	0.372	0.735
`refresh_token`	0.414	0.711	0.503	0.957
`request_password_reset`	**0.295**	0.788	0.764	0.760
`reset_password`	0.394	0.728	0.419	0.878

Table 4: Combined generation reduces per-function ε in both models. GPT-4o-mini's `request_password_reset` drops below the FLAGGED threshold (**0.295**, COMPLETE); DeepSeek V3 combined values stay above 0.30 but are uniformly lower than per-function values. When context resolves uncertainty before decoding, there is less for ε to observe.

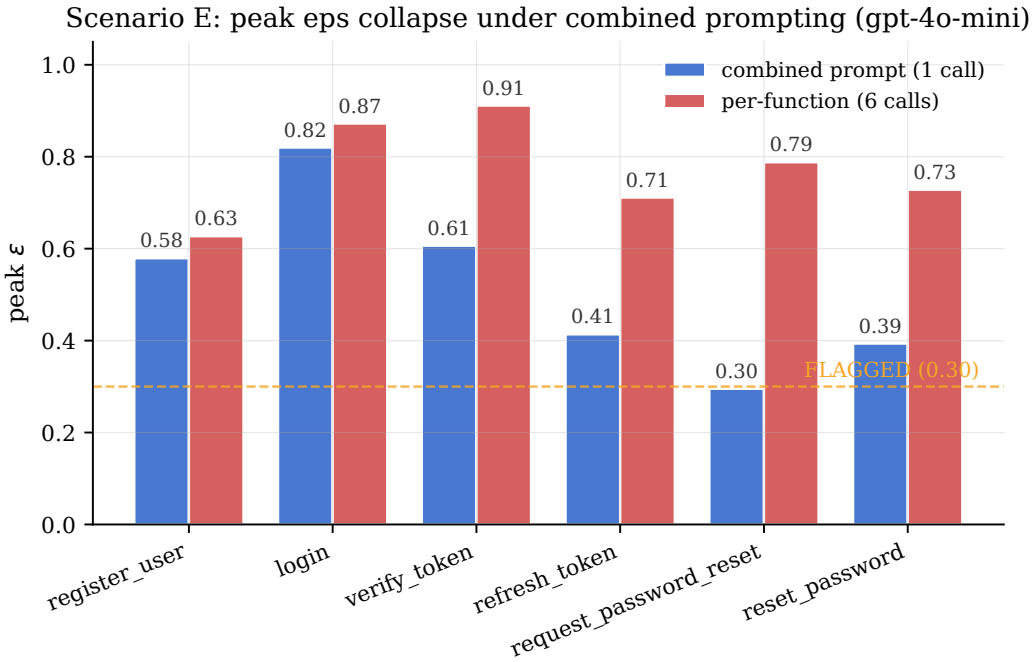

Figure 2: GPT-4o-mini: per-function generation keeps all six functions in the FLAGGED tier; combined generation collapses ε for every function, dropping one of six (`request_password_reset`, $\varepsilon = 0.295$) to COMPLETE. DeepSeek V3 shows the same reduction in combined-mode ε, but all six remain FLAGGED (Table 4).

4.3 Multi-model calibration

Moving from the per-scenario view to aggregate calibration, we evaluate a 30-prompt set: 10 LOW prompts (sort, palindrome, fibonacci, two-sum — pure logic) and 20 HIGH prompts (API/version-split tasks across Stripe, OpenAI, SQLAlchemy, FastAPI, Pydantic), run on four models. GPT-4o and DeepSeek V3 both achieve 85% HIGH detection; GPT-4o-mini and GPT-4-turbo both achieve 70%. DeepSeek V3 carries a higher LOW FP rate (70%) than the OpenAI models, consistent with its larger vocabulary of naming alternatives.

Model	HIGH detection	LOW FP
GPT-4o	85% (17/20)	50% (5/10)
GPT-4o-mini	70% (14/20)	60% (6/10)
GPT-4-turbo	70% (14/20)	60% (6/10)
DeepSeek V3	85% (17/20)	70% (7/10)

Table 5: Calibration at the FLAGGED threshold ($\varepsilon \geq 0.30$). 95% Wilson CIs: ±17pp HIGH, ±30pp LOW. Three HIGH prompts produce $\varepsilon = 0.000$ across all four models (`HTTP-GET-to-JSON`, `Pydantic serialization`, `async HTTP GET`) — *confidently-resolved domains* where training priors have converged.

4.4 LOW ground truth

All 30 LOW functions passed `exec()` assertions. Tier breakdown: COMPLETE 5, FLAGGED 20, PAUSED 5. Non-COMPLETE tiers are dominated by Type 3 (brute vs. hash two-sum; recursive vs. iterative fibonacci).

4.5 Precision/recall at tier boundaries

The calibration numbers above are per-prompt category; we now consolidate them into a single precision/recall view. On a 200-entry combined set (30 LOW exec-verified plus 170 HIGH LLM-judged; see §6.1), 53 true failures and 147 correct:

Threshold	Precision	Recall	F1
$\varepsilon \geq 0.30$ (FLAGGED+)	0.272	**1.000**	0.428
$\varepsilon \geq 0.95$ (PAUSED)	**0.309**	0.547	0.394

Table 6: $n = 200$, 53 positives; 95% Wilson CI on FLAGGED+ precision $[0.22, 0.33]$. Base failure rate 26.5% ≈ FLAGGED+ precision: the gate's value is recall, not precision. PAUSED fires rarely in our evaluation but achieves the highest precision of any tier. Recall = 1.000 is the empirical result on this 200-entry evaluated set; it does not guarantee zero misses on unseen prompts — Scenario C (GPT-4o-mini, $\varepsilon = 0.000$) shows the limit for models with converged priors. HIGH ground truth for 170/200 entries is LLM-judged by GPT-4o (§6.1); LOW ground truth (30/200) is `exec()`-verified.

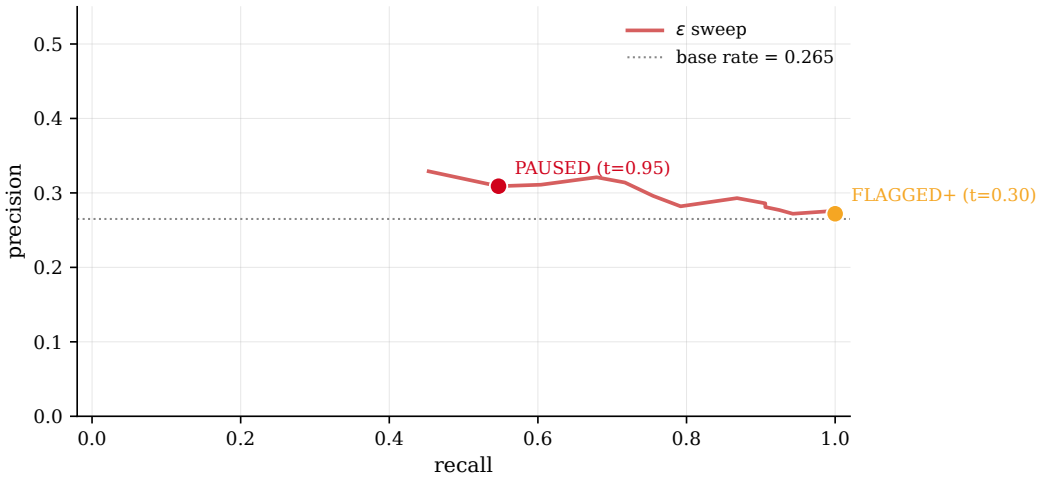

Figure 3: Precision stays near the base rate across the threshold range; recall falls monotonically. Operating points marked: FLAGGED+ ($t = 0.30$) at recall 1.00 on the evaluated set with precision barely above base rate, and PAUSED ($t = 0.95$) at recall 0.547 with the highest precision of any tier. The gate's value is recall, not precision.

Tiers are urgency signals, not calibrated probabilities. The useful binary boundary is COMPLETE vs. FLAGGED+.

4.6 Review loop: context-free vs. context-primed reviewer

Having established stage-1 recall, we turn to the stage-2 reviewer that converts the recall-maximizing filter into an end-to-end system with usable precision. Evaluated on 201 entries (production FastAPI codebase plus scenarios A–E, three models). Every FLAGGED+ entry is sent in parallel to a GPT-4o reviewer returning CLEARED/CONFIRMED; a consolidation pass processes remaining suspects. The **context-free** variant uses no reviewer context; the **context-primed** variant injects model-specific learned false-positive patterns from a prior clean run (injecting suspect patterns caused over-flagging in pilots). All results use GPT-4o as reviewer; reviewer-model ablation (e.g., with Claude or DeepSeek) is left to future work.

Metric	Context-free	Context-primed
Cleared by reviewer	107/201 (53.2%)	138/201 (68.7%)
Forwarded to consolidation	89	57
Confirmed true failures	85 (42.3%)	57 (28.4%)
Late false positives	4	0

Table 7: "Late false positive" = an entry marked CONFIRMED that was in fact correct on manual inspection. The context-primed variant is the preferred production configuration.

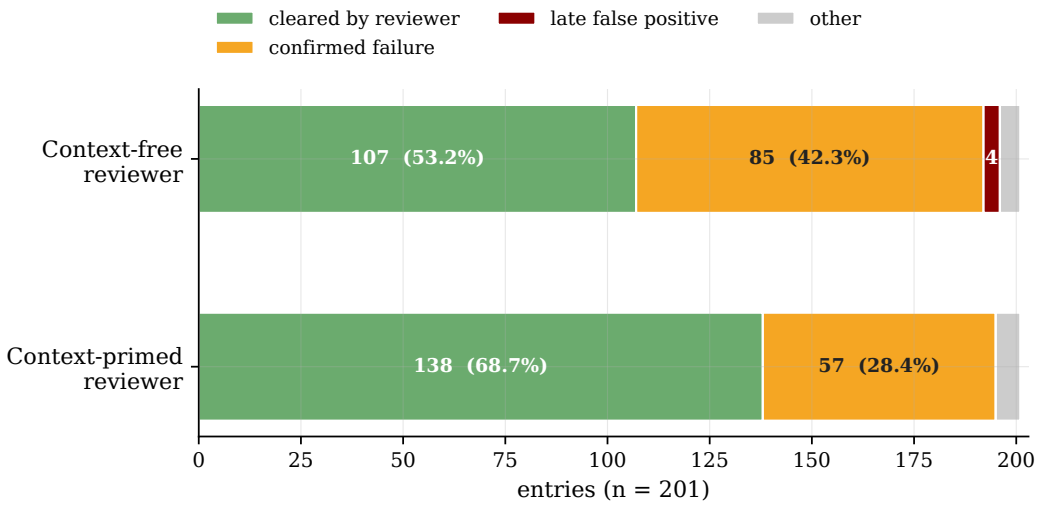

Figure 4: Injecting learned FP patterns into the reviewer (context-primed vs. context-free) shifts 31 additional entries from consolidation to cleared and eliminates late false positives, at the cost of suppressing some true failures the context-free reviewer forwarded.

End-to-end failure rates: GPT-4o-mini 40%, GPT-4o 29%, DeepSeek V3 26%. Because stage-1 recall is 100%, end-to-end precision is determined entirely by the reviewer's precision on the FLAGGED+ set. The context-primed variant's zero late-FP result rests on $n = 57$ confirmed entries and needs a larger validation study. The stage-1 recall guarantee applies to entries that reach the reviewer; entries the reviewer clears were not independently verified.

5 Comparison with Alternative Approaches

The evaluation in §4 establishes that ε fires on the prompts it is designed to catch. What it does not establish is whether ε is the right signal relative to alternatives that read the same raw logprob data, or to alternatives that use the same underlying model via a different protocol. This section provides that comparison along two axes: signal structure (what each method measures) and empirical performance on the same four-model benchmark.

5.1 Signal taxonomy

Three fundamentally different uncertainty signals are in play. **Intra-generational token-level entropy** (ε; AdaDec) reads the top-k distribution at each decoding step. **Sequence-level aggregate probability** (p_{avg}; the Spiess et al. [13] signal) averages log-probabilities over all generated tokens to produce a single scalar per function. **Inter-generational behavioural diversity** (multi-sample methods, UQLM [3]) runs the model multiple times at nonzero temperature and measures output-level agreement.

These are not three implementations of the same idea. The first two happen to read the same raw logprob data at the same 1× API cost, but aggregate it differently: ε localizes to a single high-entropy token after syntactic filtering, while p_{avg} smooths over the entire sequence. The third requires multiple model calls and measures a fundamentally different quantity: it cannot observe internal distributions at all, only the outputs those distributions produce.

5.2 p_{avg}: competitive on rate, silent on location

We compute p_{avg} directly from the `mean_logprob` fields in the same calibration runs used to produce Table 5: $p_{\text{avg}} = \exp(\text{mean_logprob})$, with higher values indicating higher confidence. For a fair comparison, we choose the p_{avg} decision threshold on each model so that the resulting LOW false-positive rate matches ε's rate on that model; we then report the HIGH detection rate achieved at that operating point.

	ε (FLAGGED+)		p_{avg} (matched FP)	
Model	HIGH det.	LOW FP	HIGH det.	LOW FP
GPT-4o	85% (17/20)	50%	85% (17/20)	50%
GPT-4o-mini	70% (14/20)	60%	75% (15/20)	10%
GPT-4-turbo	70% (14/20)	60%	90% (18/20)	40%
DeepSeek V3	85% (17/20)	70%	65% (13/20)	10%

Table 8: ε vs. p_{avg} across four models. p_{avg} ties ε on GPT-4o and outperforms ε on GPT-4o-mini and GPT-4-turbo on raw HIGH detection; on DeepSeek V3, ε is higher (85% vs. 65%). At the same $1\times$ API cost, the raw-detection picture favours neither signal uniformly.

Two facts account for the pattern. First, p_{avg} averages over all tokens, so long stretches of confident boilerplate dilute a single high-entropy decision; ε max-pools after AST filtering, so an API-selector token at $\varepsilon = 0.73$ dominates the function-level score regardless of length. On prompts where the uncertain decision is a small fraction of the generated tokens, p_{avg}'s low denominator on the long confident portion can nevertheless yield a probability low enough to fire, because even the confident tokens for some OpenAI model families carry non-trivial probability mass on alternates. Second, on naming-heavy LOW prompts, p_{avg}'s averaging suppresses the cosmetic variation that drives ε's false-positive rate above 50%, which is why some OpenAI models show higher p_{avg} LOW specificity. On DeepSeek V3, where vocabulary is larger and per-token entropy is higher on cosmetic tokens, the averaging does not save p_{avg} and ε is the better signal.

The key distinguishing property of ε is not higher detection but *localization*. p_{avg} produces one scalar per function; ε identifies that "`stripe.Charge` $P = 0.52$ vs. `stripe.PaymentIntent` $P = 0.47$ at the method-selector token" is the uncertain decision. The stage-2 reviewer uses this attribution to target its analysis: when the high-ε token is a method selector on a Stripe call, the reviewer is prompted to check for current-vs-deprecated API usage on exactly that call, not to re-read the whole function. Whether this attribution translates to better end-to-end precision in the review loop is an open question — the p_{avg} baseline has not been extended to the full 201-entry review-loop evaluation — but on the calibration benchmark the token-level output is strictly more informative at the same cost.

The older single-token `min_logprob` baseline (Table 9) is complementary to both signals. It looks at the lowest top-1 probability in the generation, which misses near-tie tokens (e.g., 0.48/0.42): the top-1 is still 0.48 and the threshold does not fire. ε sees near-uniform top-k entropy on the same token and does.

5.3 Multi-sample: structural blindspot

Inter-generational methods measure disagreement across multiple outputs of the same prompt. We ran each of the four models $N = 5$ times at temperature $T = 0.7$ on the full 30-prompt calibration benchmark, and compute output diversity by extracting the API-choice classification for each sample

Method	HIGH det. @ 30% LOW FP	HIGH det. @ 50% LOW FP
`min_logprob`	45–55%	55–65%
ε (max, AST-filtered)	**70**%	**85**%

Table 9: Matched LOW FP budgets across four models, single top-1 probability baseline. Gap is at near-tie tokens: `min_logprob` sees 0.48 and does not fire; ε sees near-uniform top-k entropy and does.

(e.g., `Charge` vs. `PaymentIntent` on Stripe) and measuring the normalized Shannon entropy over the choice distribution.

	ε (FLAGGED+)		Multi-sample N=5, T=0.7	
Model	HIGH det.	LOW FP	HIGH det.	LOW FP
GPT-4o	85% (17/20)	50%	20% (4/20)	10%
GPT-4o-mini	70% (14/20)	60%	10% (2/20)	30%
GPT-4-turbo	70% (14/20)	60%	25% (5/20)	30%
DeepSeek V3	85% (17/20)	70%	35% (7/20)	30%

Table 10: ε vs. multi-sample across four models. Multi-sample costs 5× as many API calls yet achieves 10–35% HIGH detection — 40–55pp below ε at comparable or worse LOW FP rates.

The reason multi-sample underperforms is not the sample count ($N = 5$) or the temperature ($T = 0.7$); it is structural. A model with probability 0.52 on `Charge` and 0.47 on `PaymentIntent` samples `Charge` on roughly 52% of re-rolls at $T = 1.0$, and nearly always at $T = 0.7$, where the sharpening exponent concentrates mass on the peak. The Stripe Scenario A example is a direct instance:

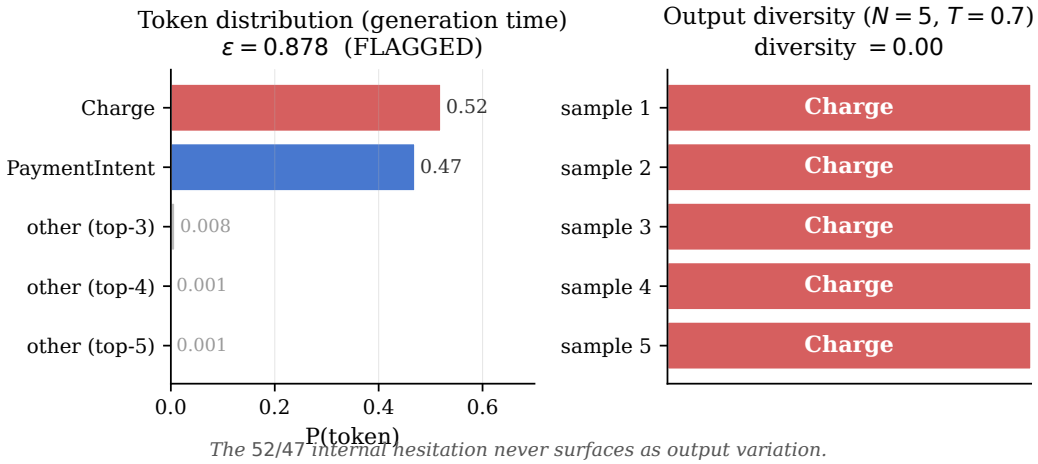

Figure 5: The intra/inter distinction made concrete on Stripe Scenario A (GPT-4o). Left: the token distribution at the method-selector token shows a near-tie (P(`Charge`) = 0.52, P(`PaymentIntent`) = 0.47), giving $\varepsilon = 0.878$. Right: five independent samples at $T = 0.7$ all emit `Charge` (diversity = 0.00). Multi-sample can only observe the right panel; the left panel is what ε reads. The 52/47 internal hesitation never surfaces as output variation.

Figure 5 is the empirical proof of the intra/inter distinction, not a theoretical assertion. GPT-4o with $\varepsilon = 0.878$ on Stripe chose `Charge` unanimously across all five samples. A stronger multi-sample check would not recover the signal: there is no output variation for a semantic-equivalence test or symbolic executor to disagree about. Our multi-sample measurement is output-level (which API was chosen); stronger equivalence checks [12] that compare code behaviour up to semantic equivalence would help on Type 3 (implementation-approach) false positives, where two semantically equivalent implementations are wrongly counted as disagreement, but they cannot recover intra-generational API-choice uncertainty that produces unanimous output.

UQLM [3] is the closest published off-the-shelf tool in this family; its `BlackBoxUQ` method samples each prompt 5 times and scores semantic disagreement via a DeBERTa NLI model. On our benchmark it detects 5% of HIGH-risk prompts at 0% LOW false-positive rate — a result driven by the same structural limit: the model is behaviourally consistent even when internally uncertain, so NLI-based clustering reports confidence ≈ 1.0 on 29 of 30 prompts.

5.4 Static analysis

Static analysis and linter rules catch *known*-bad patterns for libraries where deprecation has been encoded into a ruleset; ε fires where the model itself hesitated, including decisions not yet in any ruleset (a library updated last week, a half-migrated codebase, an internal rename). The two are complementary and address different failure modes. Static analysis is the floor under ε; ε operates in the gap where rules do not yet exist. Primary experiments use OpenAI plus DeepSeek V3 via Together AI; Anthropic exposes logprobs via streaming, and vLLM exposes the full top-k. The method requires only top-k logprobs — no model-internal access.

5.5 Summary

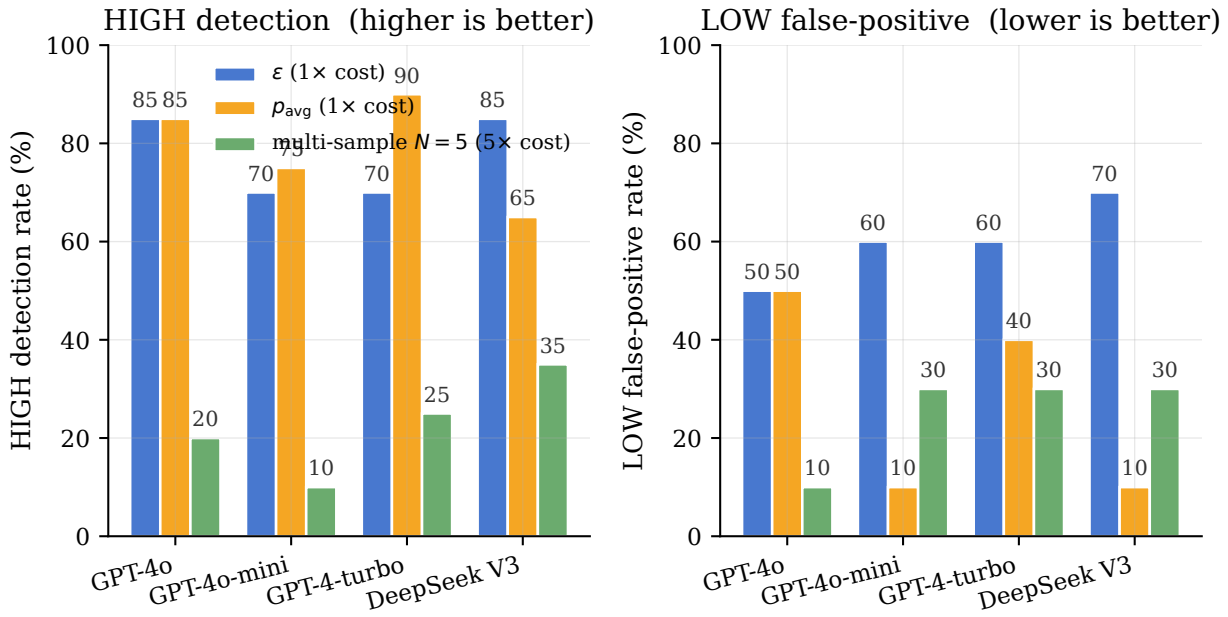

Figure 6: Three-method comparison across four models. Left: HIGH detection rate (higher is better). Right: LOW false-positive rate (lower is better). ε and p_{avg} operate at $1\times$ API cost; multi-sample at $N = 5$ costs $5\times$. p_{avg} is competitive with ε on raw detection but offers no token-level attribution; multi-sample is structurally blind to intra-generational uncertainty.

The empirical picture is honest. p_{avg} matches or outperforms ε on raw detection for three of four models at the same API cost; its gap with ε is token-level localization, not raw rate. Multi-sample, at $5\times$ the cost, reaches 10–35% detection — a structural limit, not an implementation defect.

6 Discussion

6.1 Limitations

Confident wrongness. ε cannot detect a model that is wrong and confident. If training placed all mass on a deprecated API, the token probability is sharp and ε is low; the system emits the wrong code with COMPLETE status. Scenario B is the concrete example: GPT-4o emitted `openai.ChatCompletion.create(...)` — the v0 API, which raises `AttributeError` on SDK ≥ 1.0 — with no hesitation at that token; ε fired only because of adjacent uncertainty in surrounding parameter design. Wang et al. [15] report this failure mode at scale: 25–38% deprecated-API emission on standard prompts across seven LLMs, driven by confident predictions from training data that predates the deprecation. This is a fundamental limit of any generation-time signal and a primary motivation for pairing ε with static analysis (§5.4) rather than replacing it.

Confidently-resolved domains. Prompts where every model has converged on a strong training-data prior produce $\varepsilon = 0.000$ regardless of correctness. Current-UTC datetime retrieval and HTTP-GET-to-JSON fall here across all four models. For stable widely-deployed APIs this is desirable; for deprecated-but-confidently-recalled APIs it is Scenario B's failure mode.

Small-sample calibration. Main calibration is 30 prompts per model; Wilson intervals on HIGH detection are ±17pp at $n = 20$. We claim recall-first design plus an empirical miss rate of zero on the sets evaluated, not a proof of perfect recall. The p_{avg} comparison in §5.2 uses the same 30 prompts, which bounds all statistical claims in that subsection to the same small-sample regime; cross-validation would strengthen all claims on both sides.

p_{avg} comparison scope. The p_{avg} evaluation uses the calibration benchmark (30 prompts per model, 120 total) and has not been extended to the 201-entry production-plus-scenario set used for the review-loop results. Whether p_{avg}'s better precision on some models translates to better end-to-end precision in a full two-stage system, and whether its lack of token-level attribution actually hurts reviewer performance in practice, are open empirical questions. The signal-level comparison we report should not be read as a system-level comparison; a full p_{avg}-routed review-loop experiment is future work.

Multi-sample sensitivity. The multi-sample comparison uses $N = 5$ at $T = 0.7$. Higher N or higher T might recover additional behavioural diversity on some prompts — a $T = 1.0$ run would nearly double the sampling probability of the minority branch on a 0.52/0.47 token — though the structural argument does not depend on parameter choice: ε is read at $T = 0$ where the model's internal distribution is exposed directly, while any sampling-based method is limited to what that distribution produces as behaviour. A higher-T configuration would also shift the baseline against which *generation-time* cost is compared, since production code generation is typically run at $T \leq 0.3$ or deterministically.

Type 3 floor. Implementation-approach uncertainty on simple prompts (brute vs. hash two-sum) legitimately raises ε on correct code. AST filtering does not remove it; the $\sim 50\%$ LOW FP is predominantly Type 3 and unlikely to shrink without sacrificing recall.

LLM-judged ground truth. LOW ground truth is `exec()`-verified against assertions covering return type, edge cases, and correctness. HIGH ground truth is LLM-judged via GPT-4o. Circularity exists where the evaluated model shares a family with the judge (GPT-4o-mini judged by GPT-4o); results for that model should be read with that caveat. We disclose rather than overclaim human verification we did not perform.

Hyperparameters. The cascaded tier algorithm (six checks, three thresholds) was calibrated on GPT-4o; other models inherit these. The binary FLAGGED+ gate is robust; internal tier placement is a refinement on top.

6.2 paused tier and delivery semantics

§3.5 defined PAUSED as $\varepsilon \geq 0.95$. Whether PAUSED actually holds code delivery until the review completes, or simply runs the review loop like FLAGGED, is an implementation choice we do not prescribe. In our four-model scenario sweep no HIGH prompt crossed the PAUSED threshold, so the question did not become binding. Production use would likely default to the non-blocking treatment (same routing as FLAGGED) while exposing PAUSED as a higher-urgency signal in whatever UI surfaces reviewer output.

6.3 Context learning as a microcosm

Injecting learned false-positive patterns into the reviewer's context (the configuration evaluated in §4.6) raised clearance from 53.2% to 68.7%, and late false positives fell from 4 to 0. This is in-context improvement on a fixed model; training-time improvement would be expected to compound with it. The same mechanism — priming the reviewer with model-specific priors from a prior clean run — is a candidate for the same kind of systematic study that has emerged for in-context learning more broadly.

6.4 Future work

Pipeline propagation: upstream ε should propagate as prior uncertainty into downstream generations, modulating sampling temperature or review threshold. **Open-source models:** vLLM exposes full top-k; a Llama/Qwen/Mistral calibration run would test whether the recall-first boundary holds outside OpenAI. **Per-token intervention:** surface the high-ε token as an inline editor marker, or trigger targeted re-decoding with lower temperature on just the uncertain span (AdaDec-style, but at review time). **Cross-decision consistency:** Scenario D's mismatches argue for AST-level consistency checks over ε (e.g., `async def` + blocking HTTP client). **Import-gated activation:** restricting ε scoring to functions that import external libraries (i.e., skipping pure-logic functions) would eliminate the Type 3 false-positive floor at negligible cost, since production API-integration code almost always begins with an import. **Combined $p_{\mathbf{avg}}$ + ε routing:** p_{avg}'s function-level specificity could be combined with ε's token localization — a two-signal gate that fires when both agree may recover precision without sacrificing recall. **Full-scale $p_{\mathbf{avg}}$ comparison:** extending the p_{avg} evaluation to the 201-entry review-loop set would convert the signal-level comparison of §5.2 into a system-level one.

7 Conclusion

We introduced ε, a per-token normalized Shannon entropy over top-k log-probabilities, AST-filtered and max-aggregated into a single score per generated function. ε is a filter, not a detector: it achieves the recall target established in §4.5. The contribution is the two-stage system. Pairing a recall-maximizing filter with a parallel LLM reviewer produces an architecture in which nothing uncertain ships unreviewed and the reviewer scales with generation speed.

The framing matters, and the empirical case for it is stronger than in earlier iterations of this work. Multi-sample uncertainty methods produce 10–35% detection on our benchmark — now measured directly on four models rather than inferred from a single library — because code uncertainty is intra-generational: visible in the single-generation token distribution, invisible across re-samples. The sequence-level mean token probability p_{avg}, which reads the same raw logprobs as ε, is competitive with or better than ε on raw detection rate for three of four OpenAI models at the same 1× API cost; ε's distinguishing property is not higher detection but token-level attribution, which the stage-2 reviewer uses to target its analysis. Whether that attribution translates to better end-to-end precision in a full review loop is an open question we leave for future work.

The same mechanism — early-token commitment forcing later tokens — plausibly applies in other structured-output domains (SQL, configuration files, typed program synthesis); empirical verification is left to future work. Generation is fast; review is the bottleneck. A filter that misses nothing, plus a parallel reviewer that knows where to look, moves review off the developer's critical path.

References

[1] Anastasios N Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control. In *International Conference on Learning Representations (ICLR)*, 2024.

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models, 2021.

[3] Dylan Bouchard, Mohit Singh Chauhan, David Skarbrevik, Ho-Kyeong Ra, Viren Bajaj, and Zeya Ahmad. UQLM: A python package for uncertainty quantification in large language models. *Journal of Machine Learning Research*, 2026. arXiv:2507.06196.

[4] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code generation with generated tests, 2022.

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code, 2021.

[6] Kaifeng He, Mingwei Liu, Chong Wang, Zike Li, Yanlin Wang, Xin Peng, and Zibin Zheng. Towards better code generation: Adaptive decoding with uncertainty guidance, 2025.

[7] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[8] Alain Jungo, Raphael Meier, David Alleysson, and Mauricio Reyes. Analyzing the quality and challenges of uncertainty estimations for brain tumor segmentation with deep neural networks. *Frontiers in Neuroscience*, 2020.

[9] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language models (mostly) know what they know, 2022.

[10] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *International Conference on Learning Representations (ICLR)*, 2023.

[11] Victor Quach, Adam Fisch, Tal Schuster, Adam Yala, Jae Ho Sohn, Tommi S Jaakkola, and Regina Barzilay. Conformal language modeling. In *International Conference on Learning Representations (ICLR)*, 2024.

[12] Arindam Sharma and Cristina David. Assessing correctness in LLM-based code generation via uncertainty estimation, 2025.

[13] Claudio Spiess, David Gros, Kunal Suresh Pai, Michael Pradel, Md Rafiqul Islam Rabin, Amin Alipour, Susmit Jha, Prem Devanbu, and Toufique Ahmed. Calibration and correctness of language models for code. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025. arXiv:2402.02047.

[14] Neeraj Varshney, Wenlin Yao, Hongming Zhang, Jianshu Chen, and Dong Yu. A stitch in time saves nine: Detecting and mitigating hallucinations of LLMs by validating low-confidence generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2024.

[15] Chong Wang, Kaifeng Huang, Jian Zhang, Yebo Feng, Lyuye Zhang, Yang Liu, and Xin Peng. LLMs meet library evolution: Evaluating deprecated API usage in LLM-based code completion. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025. arXiv:2406.09834.

[16] He Ye, Matias Martinez, and Martin Monperrus. SelfAPR: Self-supervised program repair with test execution diagnostics. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2022.